

1. INTRODUCTION

L'**encapsulation** est un mécanisme qui consiste à regrouper les données (attributs) et les fonctions (méthodes) qui manipulent ces données au sein d'une même unité (une classe), tout en cachant les détails internes de l'implémentation.

- **Importance pratique :**
 - Protéger l'intégrité des données (éviter qu'un code externe ne modifie un attribut de façon illogique).
 - Faciliter la maintenance : si l'implémentation interne évolue, l'interface publique reste inchangée.
 - Améliorer la lisibilité : on sait précisément quels membres de la classe sont accessibles et quels membres sont cachés.

À l'issue de cette section, vous devriez être capable :

1. De comprendre ce qu'est un attribut privé et une méthode publique dans une classe C++.
2. De déclarer et d'utiliser des getters et setters pour accéder de façon contrôlée aux données d'une classe.
3. D'expliquer pourquoi on ne rend pas toujours tous les membres d'une classe publics.

2. EXPLICATION FONDAMENTALE

1. Définition simplifiée :

- En C++, une **classe** est un plan ("blueprint") qui décrit un nouvel état de donnée (attributs) et le comportement associé (méthodes).
- **Encapsuler** signifie "mettre sous capsule" : on enferme les données sensibles pour qu'elles ne soient pas directement manipulables de l'extérieur. Seules certaines fonctions (méthodes) autorisées peuvent y toucher.

2. Analogie familière :

- Pensez à une **télécommande de télévision** : l'utilisateur appuie sur des boutons, mais il n'a pas accès aux circuits internes. Il voit seulement l'interface (boutons "Volume +", "Volume -", "Chaine suivante", etc.).
- De la même façon, dans une classe C++, on cache ("met sous coque") les attributs (internes) et on ne dévoile à l'utilisateur que les méthodes publiques (boutons) pour interagir avec la classe.

3. Les niveaux d'accès :

- **private** : accessible uniquement à l'intérieur de la classe.
- **protected** : accessible dans la classe elle-même et dans ses classes dérivées (héritage).
- **public** : accessible de n'importe où (externe au code de la classe).

Difficultés typiques :

- Confondre private et protected.
- Oublier de mettre "private:" au début, et par défaut tout est private si ce n'est pas précisé dans une classe (contrairement à struct where par défaut c'est public).
- Ne pas comprendre à quel moment utiliser protected. Pour un débutant, on se concentre surtout sur public vs private.

4. Getters et Setters (accesseurs et mutateurs) :

- **Getter** : méthode publique qui retourne la valeur d'un attribut privé (souvent nommé `getAttribut()`).
- **Setter** : méthode publique qui permet de modifier la valeur d'un attribut privé en appliquant éventuellement des contrôles (souvent nommé `setAttribut(valeur)`).
- Avantage : l'on peut vérifier la validité de la donnée avant de l'affecter (ex. refuser un solde négatif).

3. ILLUSTRATION CONCRÈTE

```
#include <iostream>
#include <string>

// Classe représentant un compte bancaire simple
class CompteBancaire {
private:
    // Attribut privé : seules les méthodes de cette classe peuvent y accéder
    double solde;
    std::string titulaire;

public:
    // Constructeur : initialise le titulaire et le solde
    CompteBancaire(const std::string& nom, double montantInitial) {
        titulaire = nom;
        if (montantInitial < 0) {
            solde = 0.0; // Si montant initial négatif, on initialise à 0
        } else {
```

```

        solde = montantInitial;
    }
}

// Getter pour solde : méthode publique qui retourne le solde actuel
double getSolde() const {
    return solde;
}

// Setter pour solde : exemple de mutateur avec vérification
void setSolde(double nouveauSolde) {
    if (nouveauSolde >= 0) {
        solde = nouveauSolde;
    } else {
        std::cout << "Erreur : solde ne peut pas être négatif." << std::endl;
    }
}

// Méthode publique pour déposer de l'argent
void deposter(double montant) {
    if (montant > 0) {
        solde += montant;
    } else {
        std::cout << "Montant de dépôt invalide." << std::endl;
    }
}

// Méthode publique pour retirer de l'argent
void retirer(double montant) {
    if (montant > 0 && montant <= solde) {
        solde -= montant;
    } else {
        std::cout << "Retrait impossible : montant invalide ou solde insuffisant." << std::endl;
    }
}

// Méthode publique pour afficher les détails du compte
void afficherDetails() const {
    std::cout << "Titulaire : " << titulaire << ", Solde : " << solde << " €" << std::endl;
}

};

int main() {
    // On crée un compte avec un solde initial de 100 €
    CompteBancaire monCompte("Alice", 100.0);

    // Affichage initial
    monCompte.afficherDetails();
    // → Titulaire : Alice, Solde : 100 €

    // On tente de définir un solde négatif (via le setter)
    monCompte.setSolde(-50.0);
    // → Erreur : solde ne peut pas être négatif.
    std::cout << "Après tentative de setter avec -50 : " << monCompte.getSolde() << " €" << std::endl;
    // → Offre : reste à 100 €

    // On dépose 30 €
    monCompte.deposer(30.0);
    std::cout << "Après dépôt de 30 : " << monCompte.getSolde() << " €" << std::endl;
    // → 130 €

    // On retire 50 €
    monCompte.retirer(50.0);
    std::cout << "Après retrait de 50 : " << monCompte.getSolde() << " €" << std::endl;
    // → 80 €

```

```

return 0;
}

```

- **Décomposition du raisonnement :**

1. **Déclaration de la classe :**

- `class CompteBancaire { ... };` crée un nouveau type `CompteBancaire`.

2. **Attributs privés :**

- `double solde;` et `std::string titulaire;` sont masqués (`private:`) : aucune autre partie du code (hors de la classe) ne peut y accéder directement.

3. **Constructeur :**

- Sert à initialiser un objet `CompteBancaire`. On vérifie que le montant initial n'est pas négatif.

4. **Getters/Setters :**

- `getSolde()` permet de lire la valeur de `solde`.
- `setSolde(double)` permet de fixer `solde` à une nouvelle valeur, après vérification (pas de solde négatif).

5. **Méthodes de gestion :**

- `deposer(double)` et `retirer(double)` modifient `solde` en vérifiant la validité des opérations.

6. **Méthode d'affichage :**

- `afficherDetails()` permet d'afficher les informations sur le compte.

7. **Utilisation dans `main()` :**

- Création d'un objet, puis tests des différentes méthodes pour voir que l'on ne peut pas contourner la logique (solde ne devient jamais négatif).

Représentation visuelle (schéma simplifié) :

Classe <code>CompteBancaire</code>	
- <code>double solde</code>	← attribut privé
- <code>string titulaire</code>	← attribut privé
+ <code>CompteBancaire(string, double)</code>	← constructeur
+ <code>double getSolde() const</code>	← getter public
+ <code>void setSolde(double)</code>	← setter public
+ <code>void deposer(double)</code>	← méthode public
+ <code>void retirer(double)</code>	← méthode public
+ <code>void afficherDetails() const</code>	← méthode public

4. VÉRIFICATION DE COMPRÉHENSION

Exercice 1.1 Quel est le rôle principal d'un attribut déclaré dans la partie `private` d'une classe ?

- A) Empêcher tout accès, même par les méthodes de la même classe.
- B) Empêcher l'accès direct depuis l'extérieur de la classe, tout en laissant les méthodes de la classe y accéder.
- C) Rendre la classe instanciable sans constructeur.
- D) Garantir que l'attribut est initialisé automatiquement à zéro.

Exercice 1.2 Dans la classe suivante, que fait la méthode `modifierValeur` ?

```

class Exemple {
private:
    int valeur;
public:
    void modifierValeur(int nouvelleValeur) {
        valeur = nouvelleValeur;
    }
    int lireValeur() const {
        return valeur;
    }
};

```

- A) Elle renvoie la valeur courante de l'attribut ``valeur``.
- B) Elle permet de modifier l'attribut ``valeur`` depuis l'extérieur, sans aucune vérification.
- C) Elle calcule une nouvelle valeur à partir de ``nouvelleValeur``.
- D) Elle empêche la modification de l'attribut ``valeur``.

Exercice 1.3 Pourquoi préférer utiliser un setter plutôt qu'un accès direct en `public` à un attribut ?

- A) Parce qu'un setter transforme automatiquement le type de la donnée.
- B) Parce qu'un setter permet d'ajouter des vérifications (par exemple refuser des valeurs invalides) avant d'affecter la donnée.
- C) Parce qu'un setter utilise moins de mémoire.
- D) Parce qu'un setter empêche complètement la lecture de la donnée.

☒ Corrections détaillées Immédiates

Exercice 1.1 – Correction

- **Réponse correcte : B)**
 - ☒ Explication : Déclarer un attribut en `private` empêche toute partie extérieure à la classe (autres fonctions, autres objets) d'accéder ou de modifier directement cet attribut. En revanche, les méthodes **de la même classe** (ou les fonctions amies si elles existent) peuvent y accéder librement.
- **Pourquoi les autres options sont fausses :**
 - A) Faux : Les méthodes de la même classe peuvent toujours lire/écrire dans un attribut `private`.
 - C) Faux : L'existence d'un attribut `private` ne rend pas la classe instanciable ou non ; la présence d'un constructeur ou non dépend des déclarations explicites.
 - D) Faux : En C++, sans initialisation explicite, un attribut non statique n'est pas automatiquement mis à zéro (il aura une valeur indéfinie).

Exercice 1.2 – Correction

- **Réponse correcte : B)**
 - ☒ Explication : La méthode `modifierValeur(int nouvelleValeur)` affecte directement `nouvelleValeur` à l'attribut privé `valeur`. Il n'y a aucune vérification (ex. pas de condition `if`). Elle permet donc un accès contrôlé (via la méthode) mais sans test.
- **Pourquoi les autres options sont fausses :**
 - A) Faux : La méthode qui renvoie la valeur courante est `lireValeur() const`, pas `modifierValeur`.
 - C) Faux : Il n'y a pas de calcul ; l'ancien contenu de `valeur` est simplement écrasé par `nouvelleValeur`.
 - D) Faux : Au contraire, elle **autorise** la modification de l'attribut `valeur`, sans contrôle.

Exercice 1.3 – Correction

- **Réponse correcte : B)**
 - ☒ Explication : Un setter (une méthode publique qui modifie un attribut privé) permet d'ajouter des **vérifications ou transformations** avant d'affecter la valeur (par exemple vérifier qu'elle n'est pas négative, qu'elle respecte un certain format, etc.). Cela protège l'intégrité de l'objet.
- **Pourquoi les autres options sont fausses :**
 - A) Faux : Un setter n'opère pas de conversion automatique de type ; il affecte simplement la valeur passée si elle est du bon type (ou convertie explicitement par le compilateur).
 - C) Faux : L'utilisation d'un setter n'altère pas la façon dont la mémoire est allouée à l'attribut.
 - D) Faux : On peut toujours **lire** l'attribut via un getter ; un setter ne bloque que l'attribution à un attribut sans contrôle.

5. RÉCAPITULATION

- **Qu'est-ce que l'encapsulation ?** C'est le fait de rendre certains membres d'une classe (attributs ou méthodes) inaccessibles depuis l'extérieur (via `private` ou `protected`), tout en fournissant des **méthodes publiques** (getters/setters) pour interagir ou lire ces données sous conditions.
- **Pourquoi l'utiliser ?**
 1. **Sécurité et fiabilité** : Empêcher un code externe d'attribuer directement une valeur invalide à un attribut.
 2. **Maintenance facilitée** : Si l'implémentation interne change, l'interface publique (signatures des méthodes) reste la même, donc le reste du code n'est pas affecté.
 3. **Lisibilité** : On sait d'emblée quels membres d'une classe sont "ouverts" à l'extérieur (publics) et lesquels sont "cachés".

CONCEPT 2 : L'ABSTRACTION

1. INTRODUCTION

L'**abstraction** est le principe qui consiste à **se focaliser sur l'essentiel** (ce que l'on doit faire) et à **cacher les détails** (comment on le fait) pour simplifier la compréhension. En C++, on réalise l'abstraction principalement via :

- Les **classes abstraites** (avec au moins une méthode `pure virtual`).
- Les **interfaces** (qu'on crée en déclarant uniquement des méthodes virtuelles pures dans une classe).
- **Importance pratique** :
 - Permet de définir un **contrat** (un ensemble de méthodes) sans révéler l'implémentation.
 - Facilite l'extensibilité : on peut ajouter de nouvelles classes qui respectent ce contrat sans toucher au code client.
 - Encourage la **séparation des responsabilités** : le code client travaille avec des signatures de méthodes, pas avec des détails techniques internes.

À l'issue de cette section, vous devriez être capable :

1. De comprendre ce qu'est une classe abstraite et pourquoi on l'utilise.
2. De déclarer une méthode purement virtuelle (`virtual ... = 0;`) dans une classe C++.
3. D'expliquer comment on utilise l'abstraction pour écrire du code générique indépendant des implémentations concrètes.

2. EXPLICATION FONDAMENTALE

1. Définition simplifiée :

- **Abstraire**, c'est **simplifier la réalité** en ne gardant que les éléments essentiels. En programmation, on crée un **contrat** (interface) qui décrit **ce que** l'on attend, sans dire **comment** c'est réalisé.
- En C++, on rend une classe **abstraite** dès qu'on déclare au moins une **méthode virtuelle pure** (notation `= 0`). On ne peut pas instancier une classe abstraite : elle sert uniquement de modèle aux classes dérivées.

2. Analogie familière :

- Imaginez que vous conduisez une voiture : vous appuyez sur un bouton "Démarrer" et la voiture démarre. Vous n'avez pas besoin de savoir comment exactement le démarreur, l'allumage, la pompe à carburant fonctionnent ensemble. L'interface "bouton Démarrer" est l'abstraction.
- De la même façon, dans un code C++, on peut demander à un objet "Calcule-moi l'aire" sans connaître la formule interne (cercle, carré, triangle, etc.).

3. Classe abstraite et méthode virtuelle pure :

- **Syntaxe d'une méthode virtuelle pure** :

```
virtual Retour nomFonction(Paramètres) = 0;
```

Exemple : `virtual double aire() const = 0;`

- Une classe qui contient **au moins une** fonction virtuelle pure est **abstraite** : on ne peut pas créer d'objet directement.
- Les **classes dérivées** doivent **implémenter** (définir) toutes les méthodes virtuelles pures pour devenir instanciables.

4. Distinguer abstraction et encapsulation :

- **Encapsulation** cache **les données** dans une classe.
- **Abstraction** cache **l'implémentation des méthodes**, en ne gardant que la signature publique.

5. Difficultés typiques :

- Oublier de définir toutes les méthodes virtuelles pures dans la classe dérivée, ce qui laisse la classe dérivée elle-même abstraite.
- Ne pas comprendre pourquoi on ne peut pas instancier une classe abstraite.
- Confondre `virtual ... = 0;` avec une mise à zéro d'un pointeur : ici, ce n'est pas une affectation, c'est une syntaxe qui signifie "pure virtual".

3. ILLUSTRATION CONCRÈTE

```
#include <iostream>
#include <cmath>
```

```

// Classe abstraite représentant la forme géométrique (forme de base)
class Forme {
public:
    // Méthode virtuelle pure ; aucune implémentation ici
    virtual double aire() const = 0;

    // On peut également ajouter un destructeur virtuel pour un nettoyage correct
    virtual ~Forme() {}
};

// Classe dérivée : Cercle
class Cercle : public Forme {
private:
    double rayon;

public:
    // Constructeur
    Cercle(double r) : rayon(r) {}

    // Implémentation de la méthode virtuelle pure "aire"
    double aire() const override {
        return M_PI * rayon * rayon;
    }
};

// Classe dérivée : Rectangle
class Rectangle : public Forme {
private:
    double largeur, hauteur;

public:
    // Constructeur
    Rectangle(double l, double h) : largeur(l), hauteur(h) {}

    // Implémentation de la méthode virtuelle pure "aire"
    double aire() const override {
        return largeur * hauteur;
    }
};

int main() {
    // On ne peut pas faire : Forme f; // ERREUR : Forme est abstraite

    // On peut manipuler un pointeur de type Forme* pour différents objets concrets
    Forme* f1 = new Cercle(5.0); // Cercle de rayon 5
    Forme* f2 = new Rectangle(4.0, 3.0); // Rectangle 4x3

    // Appeler la même méthode "aire" donne des résultats différents derrière le rideau
    std::cout << "Aire du cercle : " << f1->aire() << std::endl; // ≈78.5398
    std::cout << "Aire du rectangle : " << f2->aire() << std::endl; // 12

    // On peut stocker plusieurs formes dans un tableau de pointeurs à Forme
    Forme* tableauFormes[2];
    tableauFormes[0] = f1;
    tableauFormes[1] = f2;

    std::cout << "\nAires calculées dans un seul tableau : " << std::endl;
    for (int i = 0; i < 2; ++i) {
        std::cout << "Forme #" << i+1 << " : " << tableauFormes[i]->aire() << std::endl;
    }
    // Ceci illustre l'abstraction : on appelle aire() sans savoir comment c'est implémenté.

    // Libération de la mémoire
    delete f1;
    delete f2;
}

```

```

delete f2,
return 0;
}

```

- **Décomposition du raisonnement :**

1. **Classe abstraite `Forme` :**

- `virtual double aire() const = 0;` dit que toute classe dérivée **doit** fournir une implémentation de `aire()`.
- Si une classe dérivée n'implémente pas cette méthode, elle demeure abstraite et ne peut pas être instanciée.

2. **Classe `Cercle` :**

- Dérive de `Forme` : `class Cercle : public Forme`.
- Doit impérativement **implémenter** `double aire() const override` pour ne plus être abstraite.
- On calcule l'aire avec la formule $\pi \cdot r^2$.

3. **Classe `Rectangle` :**

- Même principe : implémentation de `aire()` pour un rectangle.

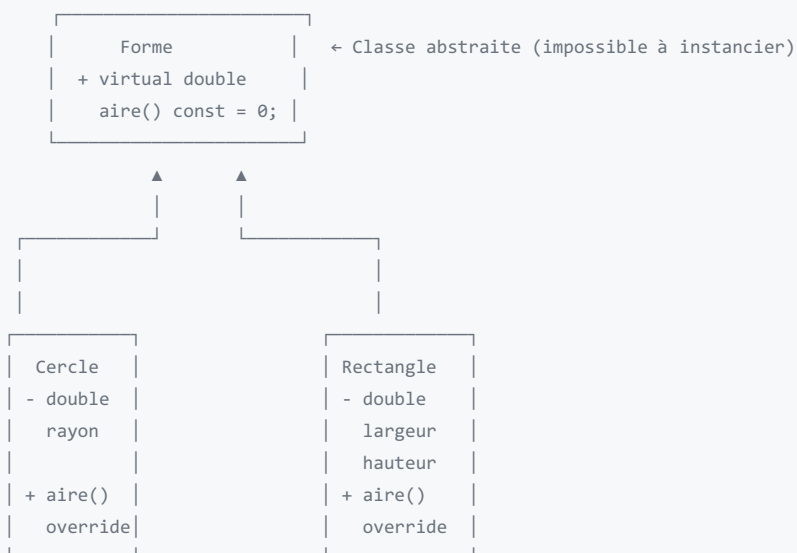
4. **Utilisation via un pointeur :**

- `Forme* f1 = new Cercle(5.0);` → On ne parle qu'en termes de "`Forme`", mais lors de l'appel à `f1->aire()`, c'est la version de `Cercle` qui s'exécute.
- C'est le **polymorphisme** rendu possible par l'abstraction.

5. **Avantage :**

- Dans le code client (ici : `main`), on ne connaît que le type abstrait `Forme`. On peut y "brancher" n'importe quelle forme dérivée tant qu'elle implémente `aire()`.

Représentation visuelle (schéma simplifié) :



4. VÉRIFICATION DE COMPRÉHENSION

Exercice 2.1 Pourquoi ne peut-on pas instancier directement la classe `Forme` ?

- Parce qu'elle ne contient pas de constructeur.
- Parce qu'elle contient au moins une méthode virtuelle pure.
- Parce qu'elle est définie dans un fichier séparé.
- Parce qu'elle n'hérite pas d'une classe de base.

Exercice 2.2 Dans la classe abstraite suivante, quelle(s) méthode(s) doivent obligatoirement être implémentée(s) par toute classe dérivée non abstraite ?

```
class Animal {
public:
    virtual void manger() const = 0;
    virtual void dormir() const = 0;
    void respirer() const {
        std::cout << "L'animal respire." << std::endl;
    }
};
```

- A) `respirer()` seulement.
- B) `manger()` et `dormir()`.
- C) Aucune : toutes les méthodes sont déjà définies.
- D) `respirer()` et `dormir()`.

Exercice 2.3 Quelle est la principale utilité de l'abstraction en C++ ?

- A) Faire gagner de la mémoire en réduisant la taille des objets.
- B) Séparer le "quoi" (interface) du "comment" (implémentation) pour permettre plus de flexibilité.
- C) Obliger le codeur à écrire uniquement des méthodes virtuelles.
- D) Remplacer complètement l'usage des classes concrètes.

☒ Corrections détaillées Immédiates

Exercice 2.1 – Correction

- Réponse correcte : B)
 - ☒ Explication : Une classe qui contient **au moins une** méthode déclarée `virtual ... = 0;` est **abstraite et ne peut pas être instanciée**. Ici, `Forme` a `virtual double aire() const = 0;`.
- Pourquoi les autres options sont fausses :
 - A) Faux : Même sans constructeur explicite, le compilateur génère un constructeur par défaut si besoin. Cela n'empêche pas l'instantiation, sauf si la classe est abstraite.
 - C) Faux : Qu'une classe soit dans un fichier séparé ou non n'influence pas sa capacité à être instanciée.
 - D) Faux : Ne pas hériter d'une classe de base n'empêche pas l'instanciation. L'élément déterminant est la présence de méthodes purement virtuelles.

Exercice 2.2 – Correction

- Réponse correcte : B)
 - ☒ Explication : `manger()` et `dormir()` sont toutes deux déclarées comme `= 0` (virtuelles pures). Toute classe dérivée non abstraite **doit** fournir une définition pour ces deux méthodes.
- Pourquoi les autres options sont fausses :
 - A) Faux : `respirer()` n'est **pas** virtuelle pure (elle est déjà implémentée). Elle n'est donc pas obligatoire à redéfinir.
 - C) Faux : Il y a deux méthodes (`manger` , `dormir`) purement virtuelles qui ne sont pas définies. On **doit** les implémenter pour que la classe dérivée ne reste pas abstraite.
 - D) Faux : `respirer()` n'est pas purement virtuelle, donc on n'est pas obligé de la redéfinir.

Exercice 2.3 – Correction

- Réponse correcte : B)
 - ☒ Explication : L'abstraction permet de définir **une interface** (le "quoi" : signatures de méthodes) et de cacher **l'implémentation** (le "comment" : code interne). Cela facilite la réutilisation, l'extensibilité, et rend le code plus flexible.
- Pourquoi les autres options sont fausses :
 - A) Faux : L'abstraction n'est pas directement liée à la taille des objets en mémoire, mais plutôt à la structure du code.
 - C) Faux : On **n'est pas obligé** d'écrire uniquement des méthodes virtuelles ; on peut mélanger méthodes concrètes et méthodes virtuelles pures.
 - D) Faux : L'abstraction ne remplace pas les classes concrètes ; elle sert plutôt de contrat que les classes concrètes dérivées implémentent.

5. RÉCAPITULATION

- **Qu'est-ce que l'abstraction ?** C'est le concept de définir **ce qu'une classe doit faire** (interface), sans préciser **comment** elle le fait, via des méthodes virtuelles pures.

- Pourquoi l'utiliser ?

1. **Flexibilité et extensibilité** : on peut ajouter de nouvelles classes dérivées sans toucher au code client.
2. **Séparation des responsabilités** : le code client s'appuie sur l'interface, pas sur l'implémentation détaillée.
3. **Polymorphisme** : appeler la même méthode sur un pointeur/une référence au type abstrait, et le bon comportement est exécuté selon la classe réelle de l'objet.

QUIZ RÉCAPITULATIF (7 – 12 questions)

Pour vérifier votre compréhension globale de l'encapsulation et de l'abstraction, répondez aux questions suivantes :

1. Qu'indique l'instruction `private` : dans une classe C++ ?

- A) Que les instructions suivantes sont générées automatiquement.
- B) Que les membres déclarés en dessous ne sont accessibles qu'à la classe elle-même (et à ses classes amies).
- C) Qu'il est possible d'accéder directement aux membres de l'extérieur sans getter.
- D) Qu'aucune méthode virtuelle ne peut y être déclarée.

2. Quelle(s) différence(s) principale(s) existe(nt) entre un champ `private` et un champ `protected` ?

- A) Un champ `private` n'est accessible qu'à sa propre classe, alors qu'un champ `protected` est accessible à la classe et à ses classes amies.
- B) Un champ `protected` peut être instancié, un champ `private` non.
- C) Les champs `protected` ont toujours une valeur par défaut, pas les `private`.
- D) Il n'y a aucune différence ; c'est une question de style.

3. Pourquoi utilise-t-on généralement des getters au lieu de déclarer un attribut `public` ?

- A) Parce qu'un getter peut effectuer des conversions de type automatiques.
- B) Parce qu'un getter permet de modifier la valeur d'un attribut.
- C) Parce qu'un getter peut valider ou transformer la valeur avant de la retourner, tout en conservant l'attribut caché.
- D) Parce qu'un getter occupe moins d'espace en mémoire.

4. Parmi les exemples suivants, laquelle représente une classe abstraite ?

- A)

```
class Animal {
    void manger() { /*...*/ }
};
```
- B)

```
class Outil {
public:
    virtual void utiliser() = 0;
};
```
- C)

```
class Vector {
public:
    int x, y;
    Vector(int a, int b) : x(a), y(b) {}
};
```
- D)

```
class Personne {
    std::string nom;
    void direBonjour() { std::cout << "Bonjour"; }
};
```

5. Si une classe dérivée n'implémente PAS toutes les méthodes virtuelles pures de sa classe de base, alors :

- A) Cette classe dérivée peut toujours être instanciée, mais les méthodes pures ne fonctionnent pas.
- B) Cette classe dérivée devient elle-même abstraite et ne peut PAS être instanciée.
- C) Le compilateur convertit automatiquement les méthodes manquantes en méthodes non abstraites.
- D) On obtient une erreur uniquement si on appelle la méthode manquante.

6. Quel est l'effet d'appeler la même méthode virtuelle (`afficher()`) sur deux classes différentes dérivées d'une même classe abstraite ?

- A) On obtient toujours le même comportement car la signature est identique.
- B) On peut obtenir deux comportements différents, car chaque dérivée fournit sa propre implémentation.
- C) Le compilateur choisit aléatoirement l'une des deux implémentations.
- D) Cela provoque une erreur à l'exécution.

7. Parmi les affirmations suivantes, laquelle est vraie concernant l'héritage et l'abstraction ?

- A) Une classe abstraite ne peut pas hériter d'une autre classe abstraite.
- B) Une classe concrète peut hériter d'une classe abstraite et fournir les implémentations manquantes.
- C) Une classe abstraite doit toujours déclarer au moins deux méthodes virtuelles pures.
- D) L'héritage n'a aucun rapport avec l'abstraction.

8. Quel est l'avantage principal de séparer l'interface (méthode virtuelle pure) de l'implémentation ?

- A) Réduire automatiquement la taille du binaire exécutable.
- B) Permettre à plusieurs développeurs de travailler sur différentes implémentations sans modifier le code client.
- C) Faire en sorte que toutes les classes dérivées utilisent la même implémentation.
- D) Imposer un style de nommage strict pour les méthodes.

9. Dans une architecture logicielle, pourquoi est-il utile de définir des classes abstraites pour représenter des concepts généraux ?

- A) Pour obfusquer le code et le rendre plus difficile à lire.
- B) Pour faciliter la réutilisation du code en garantissant que toute implémentation respecte l'interface définie.
- C) Pour obliger le programmeur à utiliser uniquement des pointeurs.
- D) Pour empêcher l'utilisation de la programmation orientée objet.

10. Quelle phrase décrit le mieux la relation entre encapsulation et abstraction ?

- A) L'encapsulation et l'abstraction sont interchangeables ; elles signifient la même chose.
- B) L'encapsulation est un moyen de mettre en œuvre l'abstraction en cachant les détails d'implémentation.
- C) L'abstraction cache les données, l'encapsulation cache les méthodes.
- D) L'encapsulation ne sert qu'à trier les attributs dans la classe.

11. **Dans l'exemple d'Animal abstrait, si on écrit :

```
class Chien : public Animal {
public:
    void dormir() const override { std::cout << "Le chien dort." << std::endl; }
};
```

Pourquoi ce code produit-il encore une classe abstraite ?

- A) Parce que la syntaxe "override" est incorrecte.
- B) Parce que la méthode "manger()" n'est pas implémentée dans "Chien".
- C) Parce que "dormir()" n'était pas dans "Animal".
- D) Parce que "Animal" avait un constructeur privé.

12. **Si on veut qu'une classe "Document" oblige les classes dérivées à implémenter ouvrir() et fermer(), on déclare :

```
class Document {
public:
    virtual void ouvrir() = 0;
    virtual void fermer() = 0;
    void imprimer() { /* code concret */ }
};
```

Quel est l'avantage principal de cette approche ?

- A) Forcer toutes les classes dérivées à avoir la même implémentation de "imprimer()".
- B) Garantir que chaque classe dérivée sait comment ouvrir et fermer, sans dire comment.
- C) Empêcher l'utilisation de pointeurs vers Document.
- D) Faire en sorte que Document soit instanciable.

CORRECTIONS DÉTAILLÉES DU QUIZ RÉCAPITULATIF

📋 Vérification Finale :

- ☒ Corrections de TOUTES les questions du quiz récapitulatif.
- ☒ Analyse de chaque option incorrecte pour chaque question.
- ☒ Explication détaillée du raisonnement pour chaque bonne réponse.
- ☒ Identification des erreurs courantes révélées par les mauvaises réponses.

- **Réponse correcte : B)**
 - **Raisonnement** : `private` : signifie que tous les membres qui suivent (attributs ou méthodes) ne sont accessibles **que depuis l'intérieur de la classe** elle-même (et ses fonctions amies).
 - **Analyse des autres options :**
 - A) Faux : `private` : ne génère aucune instruction automatique : c'est une directive d'accès.
 - C) Faux : Un attribut déclaré `private` n'est pas accessible directement de l'extérieur sans passer par une méthode publique (getter/setter).
 - D) Faux : Il n'y a pas de lien entre le fait d'avoir un membre `private` et l'impossibilité de déclarer des méthodes virtuelles. Les méthodes virtuelles peuvent être privées, protégées ou publiques.
 - **Erreur courante** : Confondre "private" avec "automatique" ou "statique" ; ou penser qu'il y a un lien avec la quantité de code généré.
-

Question 2 – Correction

- **Réponse correcte : A)**
 - **Raisonnement** : Un champ `private` est accessible **seulement à la classe** qui le déclare (et à ses fonctions amies). Un champ `protected` est accessible **à la classe et à ses classes dérivées**.
 - **Analyse des autres options :**
 - B) Faux : Ni `private` ni `protected` n'affectent la capacité d'être instancié.
 - C) Faux : Ni l'un ni l'autre ne garantit une valeur par défaut différente ; c'est la nature du type ou l'initialisation explicite qui détermine la valeur.
 - D) Faux : Il y a bien une différence d'accessibilité pour l'héritage.
 - **Erreur courante** : Penser que `protected` a un avantage mémoire ou autre, alors qu'il concerne uniquement l'héritage.
-

Question 3 – Correction

- **Réponse correcte : C)**
 - **Raisonnement** : Un getter (`getXxx()`) peut vérifier ou transformer la valeur avant de la retourner : par exemple, calculer une valeur dérivée, s'assurer qu'elle est toujours valide, etc. Cela permet de garder l'attribut sous `private` tout en contrôlant son accès.
 - **Analyse des autres options :**
 - A) Faux : Le getter ne convertit pas automatiquement le type ; si le type de l'attribut est différent, il faut gérer la conversion explicitement.
 - B) Faux : Un getter ne modifie **pas** l'attribut, il se contente généralement de le renvoyer.
 - D) Faux : L'objectif d'un getter n'est pas d'économiser de la mémoire ; il s'agit d'un mécanisme de contrôle d'accès.
 - **Erreur courante** : Croire qu'un getter agit comme un setter ou qu'il a un impact sur la mémoire.
-

Question 4 – Correction

- **Réponse correcte : B)**
 - **Raisonnement** : `class Outil { public: virtual void utiliser() = 0; };` contient une **méthode virtuelle pure** (`= 0`), donc c'est une **classe abstraite**.
 - **Analyse des autres options :**
 - A) Faux : `Animal` possède une seule méthode `manger()` définie normalement (sans `virtual ... = 0`), donc elle est **concrète** (instanciable).
 - C) Faux : `Vector` a deux attributs publics et un constructeur ; il n'y a aucune `virtual ... = 0` , donc **concrète**.
 - D) Faux : `Personne` n'a pas de `virtual ... = 0` , donc **concrète**.
 - **Erreur courante** : Confondre "virtual" avec "pure virtual" ; ou penser que toute classe avec `virtual` est abstraite alors qu'il faut `= 0` .
-

Question 5 – Correction

- **Réponse correcte : B)**
 - **Raisonnement** : Si la classe dérivée ne définit pas toutes les méthodes `virtual ... = 0` héritées, elle reste **abstraite** et donc **non instanciable**.
 - **Analyse des autres options :**
 - A) Faux : On ne peut pas instancier si des méthodes pures ne sont pas implémentées. Ce n'est pas une simple alerte à l'exécution ; c'est une **erreur à la compilation**.
 - C) Faux : Le compilateur ne comble pas automatiquement le code manquant.
 - D) Faux : Compiler refusera l'instanciation, l'erreur se produit dès la compilation, pas seulement à l'exécution.
 - **Erreur courante** : Penser que seules les méthodes appelées à l'exécution doivent être implémentées. En fait, la présence même d'une méthode pure non définie rend la classe abstraite.
-

Question 6 – Correction

- **Réponse correcte : B)**

- **Raisonnement** : Le mécanisme d'**appel virtuel** fait que, même si on a deux classes dérivées partageant le même nom de méthode `afficher()`, l'appel `ptr->afficher()` invoquera la version qui correspond au type **réel** de l'objet pointé (via la table virtuelle).

- **Analyse des autres options** :

- A) Faux : Le comportement peut différer en fonction de l'implémentation dans chaque classe dérivée.
- C) Faux : Ce n'est pas aléatoire : c'est déterminé par l'objet réel auquel pointe le pointeur.
- D) Faux : Tant que la méthode est bien virtuelle et que l'objet est valide, il n'y a pas d'erreur.

- **Erreur courante** : Croire que la même signature entraîne toujours le même code exécuté, alors que le polymorphisme permet différents comportements.
-

Question 7 – Correction

- **Réponse correcte** : B)

- **Raisonnement** : Une **classe concrète** peut hériter d'une classe abstraite et **fournir** toutes les implémentations manquantes des méthodes virtuelles pures ; ainsi, elle devient **concrète** et instanciable.

- **Analyse des autres options** :

- A) Faux : Rien n'empêche une classe abstraite d'hériter d'une autre classe abstraite.
- C) Faux : Une classe abstraite peut n'avoir qu'**une seule** méthode virtuelle pure.
- D) Faux : L'héritage est le principal vecteur permettant de faire de l'abstraction et du polymorphisme.

- **Erreur courante** : Penser qu'une classe abstraite ne peut pas être étendue, alors qu'au contraire elle est faite pour l'être.
-

Question 8 – Correction

- **Réponse correcte** : B)

- **Raisonnement** : L'intérêt est de permettre à **différents développeurs ou équipes** d'implémenter chaque version de l'interface (`ouvrir()`, `fermer()`) sans modifier le code client (par exemple, une classe `DocumentPDF`, `DocumentWord`, etc.). Le client appelle juste `ouvrir()` sur un pointeur `Document*`.

- **Analyse des autres options** :

- A) Faux : L'interface (`ouvrir() = 0`) ne contrôle pas l'implémentation de `imprimer()`.
- C) Faux : On peut très bien utiliser des pointeurs `Document*` pour manipuler les objets dynamiquement alloués.
- D) Faux : Définit `Document` comme abstraite, donc **non instanciable**.

- **Erreur courante** : Confondre abstraction d'interface avec optimisation mémoire ou limitations sur les pointeurs.
-

Question 9 – Correction

- **Réponse correcte** : B)

- **Raisonnement** : Définir une classe abstraite pour représenter un concept général (ex. `Forme`, `Animal`, `Document`) garantit que toute classe dérivée doit **respecter un ensemble de méthodes**, ce qui facilite la réutilisation et la cohérence du code (le client ne dépend que de l'interface).

- **Analyse des autres options** :

- A) Faux : L'abstraction n'existe pas pour rendre le code illisible.
- C) Faux : L'utilisation de pointeurs n'est pas une obligation, on peut aussi utiliser des références (`Animal&`).
- D) Faux : L'abstraction **est** une composante de la programmation orientée objet, pas un obstacle.

- **Erreur courante** : Penser que l'abstraction sert à "cacher" le code inutilement, alors que l'objectif est la cohérence et la flexibilité.
-

Question 10 – Correction

- **Réponse correcte** : B)

- **Raisonnement** : L'**encapsulation** vise à "mettre sous coques" les attributs et méthodes internes afin de protéger l'état de l'objet, tandis que l'**abstraction** consiste à "montrer une interface simple" en cachant les détails de l'implémentation. L'encapsulation est **un moyen** de mettre en œuvre l'abstraction : on cache les données et on expose seulement ce qui est utile.

- **Analyse des autres options** :

- A) Faux : Encapsulation et abstraction sont deux concepts distincts, bien que liés.
- C) Faux : Ce n'est pas la définition correcte : l'abstraction cache le "comment", pas forcément les données uniquement.
- D) Faux : L'encapsulation n'a pas pour rôle de "trier" les attributs ; c'est une question d'accès et de protection.

- **Erreur courante** : Confondre les deux termes ou penser qu'ils sont interchangeables.
-

Question 11 – Correction

- **Réponse correcte** : B)

- **Raisonnement** : Dans `Chien`, on n'a **pas** fourni d'implémentation pour `manger()`, qui est déclarée dans `Animal` comme `virtual void manger()`

`const = 0;` . Tant que `manger()` n'est pas implémentée, `Chien` reste **abstraite**.

- **Analyse des autres options :**
 - A) Faux : La syntaxe `override` est correcte si la signature correspond.
 - C) Faux : `dormir()` est bien dans `Animal` (déclarée en pure virtual).
 - D) Faux : La visibilité du constructeur de `Animal` n'est pas en jeu ici ; c'est la méthode pure non implémentée qui bloque.
- **Erreur courante :** Oublier qu'il y a deux méthodes pures (`manger()` ET `dormir()`) dans l'exemple d' `Animal` , et ne pas réaliser qu'il faut implémenter les deux pour rendre `Chien` concret.

Question 12 – Correction

- **Réponse correcte : B)**
 - **Raisonnement :** Déclarer `ouvrir() = 0;` et `fermer() = 0;` dans `Document` crée une **interface abstraite**. Chaque classe dérivée doit **obligatoirement** définir comment on ouvre et on ferme, sans que `Document` impose le "comment" ni le "où".
- **Analyse des autres options :**
 - A) Faux : `imprimer()` est une méthode concrète dans `Document` , mais n'a pas besoin d'être la même pour toutes les dérivées (elles peuvent donner leur propre version si elle est déclarée virtuelle).
 - C) Faux : On peut très bien utiliser un pointeur `Document*` pour manipuler dynamiquement un objet dérivé.
 - D) Faux : Au contraire, cela rend `Document` **non instanciable** (classe abstraite).
- **Erreur courante :** Penser qu'une classe abstraite rend la classe de base instanciable ou qu'elle impose l'implémentation de méthodes non virtuelles.

VÉRIFICATION FINALE

Avant de clôturer, assurons-nous :

- Toutes les corrections de chaque question de vérification (Concept 1 & Concept 2) sont bien fournies :
 - Concept 1 : Exo 1.1, 1.2, 1.3 : corrigés fournis immédiatement après chaque question.
 - Concept 2 : Exo 2.1, 2.2, 2.3 : corrigés fournis immédiatement après chaque question.
- Les corrections de toutes les questions du quiz récapitulatif (12 questions) sont présentes, avec :
 - Réponse correcte clairement identifiée.
 - Analyse de chaque option incorrecte.
 - Explication du raisonnement derrière chaque bonne réponse.
 - Identification des erreurs courantes.
- Les applications pratiques de ces connaissances (dans les récapitulatifs, nous avons indiqué : sécurité des données, maintenance facilitée, flexibilité, réutilisation du code).

APPLICATIONS PRATIQUES

1. Encapsulation :

- **Sécurisation d'un compte bancaire** (comme dans l'exemple) pour éviter qu'une opération externe ne donne un solde négatif.
- **Librairies tierces** : si vous distribuez une bibliothèque C++, vous exposez uniquement les méthodes publiques nécessaires ; le reste reste caché.
- **API** : en restreignant l'accès direct aux données internes, on évite des comportements inattendus si d'autres développeurs modifient directement les attributs.

2. Abstraction :

- **Frameworks graphiques** : définir une interface `ITexture` que chaque implémentation (DirectX, OpenGL, Vulkan) va réaliser. Le code client appelle toujours `changerTexture()` ou `dessiner()` , sans connaître la technologie en dessous.
- **Jeux vidéo** : créer une classe abstraite `Personnage` avec `virtual void attaquer() = 0;` . Les classes `Guerrier` , `Archer` , `Mage` implémentent chacune `attaquer()` différemment. Le moteur de jeu travaille avec `Personnage*` et ne se soucie pas des détails.
- **Applications Web côté serveur** : un `DataAccessObject` abstrait (DAO) déclare `virtual void ajouter(Utilisateur) = 0;` `virtual Utilisateur chercherParID(int) = 0;` . Puis on a `DAO_SQL` , `DAO_NoSQL` , etc.

Ces principes vous aideront à rendre votre code C++ plus robuste, modulable, et maintenable sur le long terme.

Bravo ! Vous venez de couvrir l'encapsulation et l'abstraction en C++. Nous avons vu :

- Comment **cacher** les données et exposer une interface minimale (encapsulation).
- Comment **spécifier** un contrat sans implémenter la logique interne (abstraction).
- Des exemples concrets (compte bancaire, formes géométriques).
- Plus d'une douzaine de questions pour tester votre compréhension, avec des **corrections détaillées** pour chaque piège potentiel.

Entraînez-vous à créer vos propres classes encapsulées et abstraites, puis essayez d'écrire un petit programme qui manipule des pointeurs vers des classes abstraites pour observer le polymorphisme en action.

▣ VÉRIFICATION DES ÉLÉMENTS OBLIGATOIRES :

- Corrections de TOUTES les questions de vérification de chaque section : ✓
- Corrections de TOUTES les questions du quiz récapitulatif : ✓
- Explication détaillée pour chaque réponse correcte et incorrecte : ✓
- Identification des applications pratiques de ces connaissances : ✓

(Ce texte a été structuré pour être lu facilement par une application de *text-to-speech*, avec des phrases concises, une numérotation claire, et des enchaînements logiques.)